Smart Budgeting and Financial Goal Management System

System Design, ERD & Data Dictionary

Project	Smart Budgeting and Financial Goal Management System — Smart Personal Budgeting and Financial Goal Management System
Prepared for	Team J
Version / Date	Version 1.0 | 8 October 2026

Working draft for project implementation. Items marked "decision required" should be confirmed by the team before development.

1. Proposed architecture

Recommended implementation shape for a manageable academic MVP:

Layer	Responsibility	Suggested option (confirm with team)
Presentation	Responsive screens, forms, charts and client-side feedback.	React + TypeScript
API / business logic	Authentication, validation, calculations, authorization and insight rules.	Django REST Framework
Persistence	Users, transactions, budgets, goals, categories and contributions.	PostgreSQL
Charts	Visualize spending by category and trends over time.	A React chart library selected by the team
Deployment	Host frontend and API; managed or hosted PostgreSQL; HTTPS.	Choose after local MVP works

2. ERD (entity relationship model)

Relationship summary: User 1—many Category, Transaction, Budget and SavingsGoal. Category 1—many Transaction and Budget. SavingsGoal 1—many GoalContribution. Each record that belongs to a user must be protected by ownership checks.

```
USER
- id (PK)
- email/username (unique)
- password_hash
- created_at
   | 1
   |----< CATEGORY (user_id FK)
   |       - id (PK)
   |       - name
   |       - kind (income/expense/both)
   |
   |----< TRANSACTION (user_id FK, category_id FK)
   |       - id (PK)
   |       - type (income/expense)
   |       - amount, transaction_date
   |       - note, created_at
   |
   |----< BUDGET (user_id FK, category_id FK)
   |       - id (PK)
   |       - period_type, start_date, end_date
   |       - limit_amount
   |
   |----< SAVINGS_GOAL (user_id FK)
          - id (PK)
          - name, target_amount
```

- target_date, status
  |
  | 1
  |----< GOAL_CONTRIBUTION (goal_id FK, user_id FK)
    - id (PK)
    - amount, contribution_date
    - note, created_at

Key constraints: use a decimal/numeric type for money, never floating-point; amounts must be positive; foreign keys must be enforced; use deletion rules carefully; unique category names may be scoped per user and category kind.

3. Data dictionary

Entity / field	Type (suggested)	Rules / meaning
User.id	UUID or BIGINT PK	Internal user identifier.
User.email	VARCHAR, unique (if email login)	Required if email is chosen as login identifier.
User.username	VARCHAR, unique (if username login)	Alternative to email; team must choose one or support both.
User.password_hash	VARCHAR	Secure hash produced by authentication framework; never plain text.
User.created_at	TIMESTAMP	Account creation time.
Category.id	PK	Category identifier.
Category.user_id	FK → User	Owner; optionally allow seeded system categories.
Category.name	VARCHAR	Required category label.
Category.kind	ENUM / VARCHAR	income, expense or both.
Transaction.id	PK	Financial record identifier.
Transaction.user_id	FK → User	Owner; authorization must verify it.
Transaction.category_id	FK → Category	Category must be valid for transaction type.
Transaction.type	ENUM / VARCHAR	income or expense.
Transaction.amount	DECIMAL(12,2)	Positive amount in selected currency.
Transaction.transaction_date	DATE	Date money was received/spent.
Transaction.note	TEXT nullable	Optional short explanation; avoid unnecessary sensitive details.
Transaction.created_at	TIMESTAMP	Record creation time.
Budget.id	PK	Budget identifier.
Budget.user_id	FK → User	Owner.
Budget.category_id	FK → Category	Category being limited.
Budget.period_type	VARCHAR	Suggested values: monthly or weekly; finalize in MVP.
Budget.start_date / end_date	DATE	Defines exact budget period; end must not precede start.
Budget.limit_amount	DECIMAL(12,2)	Positive limit.
SavingsGoal.id	PK	Goal identifier.

SavingsGoal.user_id	FK → User	Owner.
SavingsGoal.name	VARCHAR	Required goal label.
SavingsGoal.target_amount	DECIMAL(12,2)	Positive target.
SavingsGoal.target_date	DATE nullable	Optional desired completion date.
SavingsGoal.status	VARCHAR	active, paused, completed or archived; finalize permitted transitions.
GoalContribution.id	PK	Contribution identifier.
GoalContribution.goal_id	FK → SavingsGoal	Goal receiving the contribution.
GoalContribution.user_id	FK → User	Owner; must match goal owner.
GoalContribution.amount	DECIMAL(12,2)	Positive amount.
GoalContribution.contribution_date	DATE	Date contribution is recorded for.
GoalContribution.note	TEXT nullable	Optional note.

4. API outline

Resource	Example operations (illustrative)	Access rule
Auth	POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout	Public registration/login; authenticated logout.
Transactions	GET/POST /api/transactions, GET/PATCH/DELETE /api/transactions/{id}	Only the authenticated user's records.
Categories	GET/POST /api/categories, PATCH/DELETE /api/categories/{id}	Owner-scoped; seeded defaults handled explicitly.
Budgets	GET/POST /api/budgets, PATCH/DELETE /api/budgets/{id}	Owner-scoped; validate period/category.
Goals	GET/POST /api/goals, PATCH/DELETE /api/goals/{id}	Owner-scoped.
Contributions	GET/POST /api/goals/{id}/contributions	Goal and contribution must belong to same user.
Dashboard/reports	GET /api/dashboard?start=...&end=..., GET /api/reports/...	Authenticated; calculated from owned records.

5. Security and integrity design

- Use a mature authentication library/framework rather than inventing password handling.
- Perform authorization on the server for every object request; hiding UI links is not security.
- Validate all input on the server, including amounts, dates, categories and period boundaries.
- Use HTTPS in deployment; keep secrets in environment variables and out of Git.
- Use database migrations and foreign-key constraints; avoid hard deletion of related records without a clear policy.
- Do not log passwords, tokens or full financial records unnecessarily.

- Prepare test data with fictional amounts, not real sensitive personal financial information.

6. Decisions to settle before implementation

- Confirm React/TypeScript + Django REST Framework + PostgreSQL or choose another stack.
- Choose email, username or both for authentication.
- Confirm single currency (suggested ZMW for MVP).
- Decide category ownership and whether default categories are system-wide or user-specific.
- Define budget period behavior and duplicate-budget policy.
- Define how savings contributions relate to transaction records to avoid double-counting money.
- Choose delete/archive behavior for goals and categories.
- Agree insight rules and minimum data needed before an insight appears.